

Generative Bayesian Classifiers, Fisher Discriminants, and Multiclass Subspace Projections

A Unified Mathematical, Information-Theoretic, and Physical Foundation of Linear Discriminant Analysis (LDA)

Contents

1	The Generative Learning Paradigm and Bayes Decision Theory	2
1.1	Bayes' Theorem and Optimal Decision Boundaries	2
2	Gaussian Discriminant Analysis and Linear Decision Boundaries	2
2.1	The Homoscedasticity Assumption	3
2.2	Derivation of the Linear Discriminant Function	3
2.3	Equation of the Decision Hyperplane	3
3	Geometric Interpretation and Mahalanobis Metric	4
3.1	Mahalanobis Distance vs. Euclidean Distance	4
3.2	Spherical Whitening Transformation	4
4	Fisher's Linear Discriminant	4
4.1	Scatter Matrices and Separation Criterion	5
4.2	Optimization and Solution via Generalized Eigenvalue Problem	6
5	Multi-Class LDA and Dimensionality Reduction	6
5.1	Multi-Class Scatter Matrix Decompositions	6
5.2	Subspace Projection and Rank Bounds	7
5.3	Comparison: PCA vs. LDA	7
6	Physical Analogies: Mechanics and Coupled Oscillators	8
6.1	Center of Mass and Moment of Inertia Tensor	8
6.2	Coupled Harmonic Oscillators and Normal Modes	8
7	Parameter Estimation, Regularization, and Shrinkage	8
7.1	Maximum Likelihood Parameter Estimates	8
7.2	Small-Sample Singularities: Regularized Discriminant Analysis (RDA)	9
8	Production-Grade Vectorized NumPy Implementation	9

1 The Generative Learning Paradigm and Bayes Decision Theory

In statistical learning, discriminative modeling approaches (such as Logistic Regression and Support Vector Machines) model the conditional class probability $P(Y = k \mid \mathbf{X} = \mathbf{x})$ directly without attempting to model the underlying feature distribution $P(\mathbf{X} = \mathbf{x})$.

In contrast, **Generative Learning Algorithms** construct a full probabilistic model of the data-generating process by specifying both the prior distribution of classes $P(Y = k)$ and the class-conditional density $P(\mathbf{X} = \mathbf{x} \mid Y = k)$.

Definition 1.1 (Generative Modeling Framework). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be an i.i.d. dataset of N observations drawn from an unknown joint distribution $p_{\text{data}}(\mathbf{x}, y)$. Each input vector $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is a d -dimensional feature vector, and $y^{(i)} \in \{1, 2, \dots, K\}$ denotes one of K discrete categorical classes. A generative model specifies:

1. Categorical Class Priors: $\pi_k = P(Y = k)$, subject to $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$.
2. Class-Conditional Densities: $f_k(\mathbf{x}) = p(\mathbf{X} = \mathbf{x} \mid Y = k)$.

1.1 Bayes' Theorem and Optimal Decision Boundaries

Using Bayes' Theorem, the posterior probability of class membership given an observation $\mathbf{x}$ is formulated as:

$$P(Y = k \mid \mathbf{X} = \mathbf{x}) = \frac{p(\mathbf{X} = \mathbf{x} \mid Y = k)P(Y = k)}{p(\mathbf{X} = \mathbf{x})} = \frac{f_k(\mathbf{x})\pi_k}{\sum_{l=1}^K f_l(\mathbf{x})\pi_l} \quad (1)$$

According to **Bayes Decision Theory**, optimal classification minimizing the overall expected classification error (Bayes Risk under 0-1 loss) assigns an observation $\mathbf{x}$ to the class that maximizes the posterior probability:

$$\hat{y}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} P(Y = k \mid \mathbf{X} = \mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} f_k(\mathbf{x})\pi_k \quad (2)$$

Monotonic transformations preserve inequalities. Taking the natural logarithm yields the equivalent log-posterior decision rule:

$$\hat{y}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} [\log f_k(\mathbf{x}) + \log \pi_k] \quad (3)$$

The decision boundary separating any two classes k and l corresponds to the geometric locus of points where their posterior probabilities are equal:

$$\left\{ \mathbf{x} \in \mathbb{R}^d \mid P(Y = k \mid \mathbf{X} = \mathbf{x}) = P(Y = l \mid \mathbf{X} = \mathbf{x}) \right\} \iff \log f_k(\mathbf{x}) + \log \pi_k = \log f_l(\mathbf{x}) + \log \pi_l \quad (4)$$

2 Gaussian Discriminant Analysis and Linear Decision Boundaries

Linear Discriminant Analysis (LDA) arises when we assume that each class-conditional density $f_k(\mathbf{x})$ follows a d -dimensional **Multivariate Gaussian (Normal) Distribution**:

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right) \quad (5)$$

where $\boldsymbol{\mu}_k \in \mathbb{R}^d$ is the mean vector of class k , and $\Sigma_k \in \mathbb{R}^{d \times d}$ is the symmetric positive-definite class-covariance matrix.

2.1 The Homoscedasticity Assumption

The fundamental assumption distinguishing Linear Discriminant Analysis (LDA) from Quadratic Discriminant Analysis (QDA) is **Homoscedasticity**—the assumption that all K classes share a common covariance matrix Σ :

$$\Sigma_1 = \Sigma_2 = \dots = \Sigma_K = \Sigma \quad (6)$$

Substituting the homoscedastic multivariate Gaussian density into the log-posterior decision objective yields:

$$\begin{aligned} \log(f_k(\mathbf{x})\pi_k) &= \log f_k(\mathbf{x}) + \log \pi_k \\ &= \log \left(\frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \right) - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k \end{aligned} \quad (7)$$

Expanding the quadratic vector-matrix-vector term:

$$(\mathbf{x} - \boldsymbol{\mu}_k)^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_k + \boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k \quad (8)$$

Note that $\mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_k = \boldsymbol{\mu}_k^T \Sigma^{-1} \mathbf{x}$ because Σ^{-1} is symmetric.

Substituting this expansion back into the log-posterior expression:

$$\log(f_k(\mathbf{x})\pi_k) = -\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x} + \mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k - \frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| \quad (9)$$

2.2 Derivation of the Linear Discriminant Function

When comparing classes k and l during the argmax evaluation, terms independent of class index k (specifically, $-\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x}$, $-\frac{d}{2} \log(2\pi)$, and $-\frac{1}{2} \log |\Sigma|$) are constant across all candidate classes. Removing these additive invariants isolates the **Linear Discriminant Function** $\delta_k(\mathbf{x})$:

Definition 2.1 (LDA Linear Discriminant Function).

$$\delta_k(\mathbf{x}) = \mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k \quad (10)$$

Notice that $\delta_k(\mathbf{x})$ is strictly a **linear function** of the feature vector $\mathbf{x}$:

$$\delta_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k \quad (11)$$

where the weight vector $\mathbf{w}_k \in \mathbb{R}^d$ and bias term $b_k \in \mathbb{R}$ are defined as:

$$\mathbf{w}_k = \Sigma^{-1} \boldsymbol{\mu}_k, \quad b_k = -\frac{1}{2}\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k \quad (12)$$

2.3 Equation of the Decision Hyperplane

The decision boundary separating class k from class l is determined by the equation $\delta_k(\mathbf{x}) = \delta_l(\mathbf{x})$:

$$\mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k = \mathbf{x}^T \Sigma^{-1} \boldsymbol{\mu}_l - \frac{1}{2}\boldsymbol{\mu}_l^T \Sigma^{-1} \boldsymbol{\mu}_l + \log \pi_l \quad (13)$$

Rearranging terms into a single linear equation:

$$\mathbf{x}^T \Sigma^{-1} (\boldsymbol{\mu}_k - \boldsymbol{\mu}_l) - \frac{1}{2} (\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k - \boldsymbol{\mu}_l^T \Sigma^{-1} \boldsymbol{\mu}_l) + \log \left(\frac{\pi_k}{\pi_l} \right) = 0 \quad (14)$$

This represents an explicit d -dimensional hyperplane equation $\mathbf{a}^T \mathbf{x} + c = 0$, where:

$$\mathbf{a} = \Sigma^{-1} (\boldsymbol{\mu}_k - \boldsymbol{\mu}_l), \quad c = -\frac{1}{2} (\boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k - \boldsymbol{\mu}_l^T \Sigma^{-1} \boldsymbol{\mu}_l) + \log \left(\frac{\pi_k}{\pi_l} \right) \quad (15)$$

Remark 2.1 (Comparison with QDA). If we do not enforce homoscedasticity ($\Sigma_k \neq \Sigma_l$), the quadratic terms $\mathbf{x}^T \Sigma_k^{-1} \mathbf{x}$ do not cancel out. The discriminant function retains a quadratic term $\mathbf{x}^T \mathbf{A}_k \mathbf{x}$, resulting in **Quadratic Discriminant Analysis (QDA)** with curved quadratic decision hyper-quadratics.

3 Geometric Interpretation and Mahalanobis Metric

The linear decision boundary of LDA admits a clean geometric interpretation in terms of vector metrics in Riemannian spaces.

3.1 Mahalanobis Distance vs. Euclidean Distance

The standard Euclidean distance metric $\|\mathbf{x} - \boldsymbol{\mu}\|_2 = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T(\mathbf{x} - \boldsymbol{\mu})}$ assumes isotropic variance across all spatial directions. When feature correlations exist or scales differ, Euclidean distances distort geometric relationships.

Definition 3.1 (Mahalanobis Distance). Given a positive-definite covariance matrix $\boldsymbol{\Sigma}$, the squared **Mahalanobis Distance** between a vector $\mathbf{x}$ and a mean vector $\boldsymbol{\mu}_k$ is:

$$D_M^2(\mathbf{x}, \boldsymbol{\mu}_k) = (\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \quad (16)$$

Rewriting the linear discriminant function $\delta_k(\mathbf{x})$ in terms of Mahalanobis distance:

$$\begin{aligned} -\frac{1}{2} D_M^2(\mathbf{x}, \boldsymbol{\mu}_k) + \log \pi_k &= -\frac{1}{2} (\mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x} - 2\mathbf{x}^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k) + \log \pi_k \\ &= -\frac{1}{2} \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x} + \delta_k(\mathbf{x}) \end{aligned} \quad (17)$$

Because $-\frac{1}{2} \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x}$ is invariant to class index k :

$$\arg \max_k \delta_k(\mathbf{x}) \equiv \arg \min_k \left[\frac{1}{2} D_M^2(\mathbf{x}, \boldsymbol{\mu}_k) - \log \pi_k \right] \quad (18)$$

If priors are uniform ($\pi_1 = \pi_2 = \dots = \pi_K = 1/K$), LDA assigns $\mathbf{x}$ to the class with the smallest Mahalanobis distance to its mean $\boldsymbol{\mu}_k$.

3.2 Spherical Whitening Transformation

Because $\boldsymbol{\Sigma}$ is symmetric positive-definite, its eigendecomposition is $\boldsymbol{\Sigma} = \mathbf{V} \boldsymbol{\Lambda} \mathbf{V}^T$, where $\mathbf{V}$ is an orthogonal matrix of eigenvectors and $\boldsymbol{\Lambda}$ is a diagonal matrix of positive eigenvalues.

We define the **Spherical Whitening Matrix** $\boldsymbol{\Sigma}^{-1/2} = \mathbf{V} \boldsymbol{\Lambda}^{-1/2} \mathbf{V}^T$. Defining transformed coordinates $\mathbf{x}^* = \boldsymbol{\Sigma}^{-1/2} \mathbf{x}$ and $\boldsymbol{\mu}_k^* = \boldsymbol{\Sigma}^{-1/2} \boldsymbol{\mu}_k$:

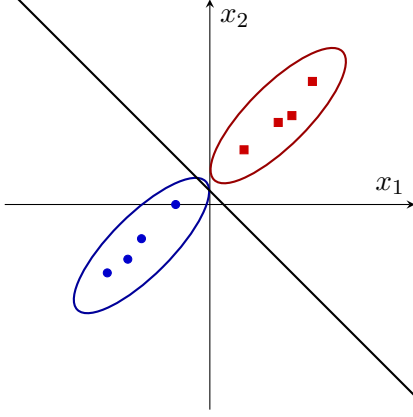
$$D_M^2(\mathbf{x}, \boldsymbol{\mu}_k) = (\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}^{-1/2} \boldsymbol{\Sigma}^{-1/2} (\mathbf{x} - \boldsymbol{\mu}_k) = \|\mathbf{x}^* - \boldsymbol{\mu}_k^*\|_2^2 \quad (19)$$

In the whitened coordinate space $\mathbf{x}^*$, the shared covariance matrix becomes the identity matrix $\mathbf{I}_d$. The class distributions become isotropic spherical Gaussians, and decision boundaries become orthogonal bisecting hyperplanes between class centroids $\boldsymbol{\mu}_k^*$ and $\boldsymbol{\mu}_l^*$.

4 Fisher's Linear Discriminant

An alternative non-parametric derivation of LDA was formulated by Sir Ronald Fisher in 1936. Rather than assuming Gaussian distributions, Fisher sought a linear projection vector $\mathbf{w} \in \mathbb{R}^d$ that maps high-dimensional data $\mathbf{x} \in \mathbb{R}^d$ onto a 1D scalar line $y = \mathbf{w}^T \mathbf{x}$ such that class separability is maximized.

Raw Feature Space (Correlated Σ)



Whitened Space ($\mathbf{x}^* = \Sigma^{-1/2}\mathbf{x}$)

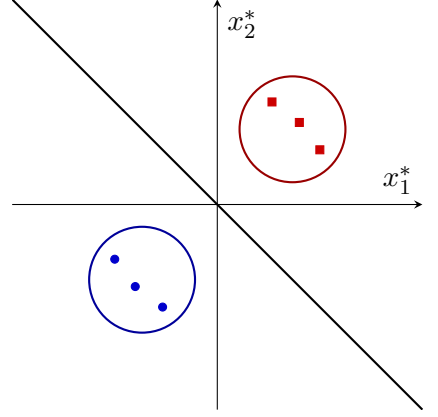


Figure 1: In the whitened space $\mathbf{x}^*$, correlated elliptical distributions map to isotropic spheres, converting Mahalanobis distance decisions into perpendicular Euclidean bisectors.

4.1 Scatter Matrices and Separation Criterion

Consider a binary classification problem with sample means $\mathbf{m}_1, \mathbf{m}_2 \in \mathbb{R}^d$ and sample counts N_1, N_2 :

$$\mathbf{m}_1 = \frac{1}{N_1} \sum_{i \in C_1} \mathbf{x}^{(i)}, \quad \mathbf{m}_2 = \frac{1}{N_2} \sum_{i \in C_2} \mathbf{x}^{(i)} \quad (20)$$

After projection onto $\mathbf{w}$, the projected scalar means are $\tilde{m}_1 = \mathbf{w}^T \mathbf{m}_1$ and $\tilde{m}_2 = \mathbf{w}^T \mathbf{m}_2$. Distance between projected means is $(\tilde{m}_1 - \tilde{m}_2)^2 = (\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2))^2$.

Simply maximizing difference between projected means fails if projected class variances are large, causing class overlap. Fisher defined class variance in projected space as:

$$\tilde{s}_k^2 = \sum_{i \in C_k} (y^{(i)} - \tilde{m}_k)^2 = \sum_{i \in C_k} (\mathbf{w}^T \mathbf{x}^{(i)} - \mathbf{w}^T \mathbf{m}_k)^2 \quad (21)$$

Definition 4.1 (Scatter Matrices). We define the ****Within-Class Scatter Matrix**** S_W and ****Between-Class Scatter Matrix**** S_B :

$$S_W = S_1 + S_2 = \sum_{i \in C_1} (\mathbf{x}^{(i)} - \mathbf{m}_1)(\mathbf{x}^{(i)} - \mathbf{m}_1)^T + \sum_{i \in C_2} (\mathbf{x}^{(i)} - \mathbf{m}_2)(\mathbf{x}^{(i)} - \mathbf{m}_2)^T \quad (22)$$

$$S_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \quad (23)$$

Expressing projected variance terms using scatter matrices:

$$\tilde{s}_1^2 + \tilde{s}_2^2 = \mathbf{w}^T S_W \mathbf{w} \quad (24)$$

$$(\tilde{m}_1 - \tilde{m}_2)^2 = (\mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2))^2 = \mathbf{w}^T (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w} = \mathbf{w}^T S_B \mathbf{w} \quad (25)$$

Definition 4.2 (Fisher Criterion / Rayleigh Quotient). Fisher's objective minimizes within-class spread while maximizing between-class distance:

$$J(\mathbf{w}) = \frac{(\tilde{m}_1 - \tilde{m}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} \quad (26)$$

4.2 Optimization and Solution via Generalized Eigenvalue Problem

To maximize $J(\mathbf{w})$, we differentiate with respect to $\mathbf{w}$ using the quotient rule:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{(2S_B \mathbf{w})(\mathbf{w}^T S_W \mathbf{w}) - (\mathbf{w}^T S_B \mathbf{w})(2S_W \mathbf{w})}{(\mathbf{w}^T S_W \mathbf{w})^2} = \mathbf{0} \quad (27)$$

Dividing by $2(\mathbf{w}^T S_W \mathbf{w})^{-1}$:

$$S_B \mathbf{w} - \left(\frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} \right) S_W \mathbf{w} = \mathbf{0} \implies S_B \mathbf{w} = \lambda S_W \mathbf{w} \quad (28)$$

where $\lambda = J(\mathbf{w})$ is a scalar equal to Rayleigh Quotient value.

This is a ****Generalized Eigenvalue Problem****:

$$S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w} \quad (29)$$

Since $S_B \mathbf{w} = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w}$, the vector $S_B \mathbf{w}$ points in direction of $(\mathbf{m}_1 - \mathbf{m}_2)$ because $(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w}$ is a scalar. Thus:

$$S_W^{-1} S_B \mathbf{w} = S_W^{-1} \left((\mathbf{m}_1 - \mathbf{m}_2) \underbrace{(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w}}_{\text{scalar } c} \right) = c S_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2) \quad (30)$$

Since direction of projection vector $\mathbf{w}$ matters rather than scale, we drop scalar c :

$$\mathbf{w}^* \propto S_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2) \quad (31)$$

Theorem 4.1 (Equivalence of Fisher Criterion and Probabilistic LDA). When sample covariance estimates equal within-class scatter ($\Sigma \propto S_W$), Fisher's optimal projection direction $\mathbf{w}^* = S_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2)$ is identical to vector coefficient direction $\Sigma^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ derived from probabilistic Gaussian LDA.

5 Multi-Class LDA and Dimensionality Reduction

Fisher's discriminant extends to $K > 2$ classes, serving as a supervised dimensionality reduction method.

5.1 Multi-Class Scatter Matrix Decompositions

Let $N = \sum_{k=1}^K N_k$ be sample size, $\mathbf{m}_k$ be mean of class k , and $\mathbf{m}$ be global sample mean:

$$\mathbf{m} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}^{(i)} = \frac{1}{N} \sum_{k=1}^K N_k \mathbf{m}_k \quad (32)$$

Definition 5.1 (Total, Within-Class, and Between-Class Scatter).

$$S_T = \sum_{i=1}^N (\mathbf{x}^{(i)} - \mathbf{m})(\mathbf{x}^{(i)} - \mathbf{m})^T \quad (33)$$

$$S_W = \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}^{(i)} - \mathbf{m}_k)(\mathbf{x}^{(i)} - \mathbf{m}_k)^T \quad (34)$$

$$S_B = \sum_{k=1}^K N_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T \quad (35)$$

Theorem 5.1 (Scatter Matrix Decomposition). Total scatter matrix decomposes into sum of within-class and between-class scatter matrices:

$$S_T = S_W + S_B \quad (36)$$

Proof.

$$\begin{aligned} S_T &= \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}^{(i)} - \mathbf{m}_k + \mathbf{m}_k - \mathbf{m})(\mathbf{x}^{(i)} - \mathbf{m}_k + \mathbf{m}_k - \mathbf{m})^T \\ &= \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}^{(i)} - \mathbf{m}_k)(\mathbf{x}^{(i)} - \mathbf{m}_k)^T + \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T \\ &\quad + \underbrace{\sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}^{(i)} - \mathbf{m}_k)(\mathbf{m}_k - \mathbf{m})^T}_{\mathbf{0}} + \sum_{k=1}^K (\mathbf{m}_k - \mathbf{m}) \underbrace{\sum_{i \in C_k} (\mathbf{x}^{(i)} - \mathbf{m}_k)^T}_{\mathbf{0}} \\ &= S_W + \sum_{k=1}^K N_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T = S_W + S_B \end{aligned} \quad (37)$$

□

5.2 Subspace Projection and Rank Bounds

To reduce feature space from d dimensions to $p < d$ dimensions ($p \leq K - 1$), we construct a projection matrix $\mathbf{W} \in \mathbb{R}^{d \times p}$. Projected data $\mathbf{y} = \mathbf{W}^T \mathbf{x}$ has scatter matrices $\tilde{S}_W = \mathbf{W}^T S_W \mathbf{W}$ and $\tilde{S}_B = \mathbf{W}^T S_B \mathbf{W}$.

Fisher's multi-class criterion maximizes determinant ratio (volume ratio of scatters):

$$J(\mathbf{W}) = \frac{|\mathbf{W}^T S_B \mathbf{W}|}{|\mathbf{W}^T S_W \mathbf{W}|} \quad (38)$$

Columns of optimal projection matrix $\mathbf{W}^*$ are leading eigenvectors corresponding to largest eigenvalues of generalized eigensystem:

$$S_W^{-1} S_B \mathbf{w}_j = \lambda_j \mathbf{w}_j \quad (39)$$

Property 5.1 (Rank Bound of Subspace). Since $S_B = \sum_{k=1}^K N_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T$ is sum of K rank-1 outer products subject to constraint $\sum N_k (\mathbf{m}_k - \mathbf{m}) = \mathbf{0}$, $\text{rank}(S_B) \leq K - 1$.

Consequently, $S_W^{-1} S_B$ has at most $K - 1$ non-zero eigenvalues. LDA can project data to at most $p_{\max} = \min(d, K - 1)$ dimensions.

5.3 Comparison: PCA vs. LDA

Property	Principal Component Analysis (PCA)	Linear Discriminant Analysis (LDA)
Learning Type	Unsupervised	Supervised
Objective	Maximize total variance S_T	Maximize ratio S_B/S_W
Scatter Focus	Ignores class labels entirely	Separates class means while shrinking variance
Max Dimensions	$\min(N, d)$	$\min(d, K - 1)$
Sensitivity	Dominant variance directions (may mix classes)	Class discriminative directions

Table 1: Structural differences between PCA and LDA.

6 Physical Analogies: Mechanics and Coupled Oscillators

6.1 Center of Mass and Moment of Inertia Tensor

LDA mathematics maps directly onto classical rigid-body mechanics. Consider K physical clouds of particles in d -dimensional space, where each particle i has unit mass $m_i = 1$:

- **Class Centroid $\boldsymbol{\mu}_k$** represents **Center of Mass** of k -th cluster:

$$\mathbf{r}_{\text{cm}}^{(k)} = \frac{1}{M_k} \sum_{i \in C_k} m_i \mathbf{r}_i = \boldsymbol{\mu}_k \quad (40)$$

- **Within-Class Scatter S_W** corresponds to sum of **Moment of Inertia Tensors** $I^{(k)}$ of individual particle clouds evaluated about their respective centers of mass:

$$I_{ij}^{(k)} = \sum_{\alpha \in C_k} \left(r_{\alpha,i} - r_{\text{cm},i}^{(k)} \right) \left(r_{\alpha,j} - r_{\text{cm},j}^{(k)} \right) \implies S_W = \sum_{k=1}^K I^{(k)} \quad (41)$$

- **Between-Class Scatter S_B** corresponds to orbital moment of inertia tensor of system treating each cluster as point mass N_k located at its center of mass $\mathbf{r}_{\text{cm}}^{(k)}$ relative to global system center of mass $\mathbf{r}_{\text{cm}}$.

Parallel Axis Theorem (Steiner's Theorem) states total inertia tensor equals internal inertia plus orbital inertia, matching scatter decomposition $S_T = S_W + S_B$.

6.2 Coupled Harmonic Oscillators and Normal Modes

The generalized eigenvalue problem $S_B \mathbf{w} = \lambda S_W \mathbf{w}$ matches the physical equation for **Normal Modes of Vibration** in coupled mechanical/molecular systems:

$$\mathbf{K} \mathbf{q}_j = \omega_j^2 \mathbf{M} \mathbf{q}_j \quad (42)$$

where $\mathbf{K}$ is potential energy stiffness matrix, $\mathbf{M}$ is kinetic energy mass/inertia matrix, $\mathbf{q}_j$ is normal mode eigenvector, and ω_j^2 is eigenfrequency.

- S_B acts as **Stiffness Matrix $\mathbf{K}$** measuring restoring forces between cluster centers.
- S_W acts as **Mass Matrix $\mathbf{M}$** measuring inertia/internal variance resisting motion.
- LDA projection vectors $\mathbf{w}_j$ are **Normal Coordinate Transformations** decoupling system into independent resonant modes ordered by discriminative frequency λ_j .

7 Parameter Estimation, Regularization, and Shrinkage

7.1 Maximum Likelihood Parameter Estimates

Under Gaussian assumptions, parameters $\boldsymbol{\theta} = \{\pi_1, \dots, \pi_K, \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}\}$ are estimated via MLE:

$$\hat{\pi}_k = \frac{N_k}{N} \quad (43)$$

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{i \in C_k} \mathbf{x}^{(i)} \quad (44)$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N - K} \sum_{k=1}^K \sum_{i \in C_k} (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)^T = \frac{1}{N - K} S_W \quad (45)$$

Division by $N - K$ provides an unbiased estimate of pooled covariance matrix $\boldsymbol{\Sigma}$.

7.2 Small-Sample Singularities: Regularized Discriminant Analysis (RDA)

When feature dimension d exceeds sample size N ($d > N$), S_W has rank at most $N - K < d$. S_W becomes singular and non-invertible, causing matrix inversion S_W^{-1} to fail.

To resolve singularity, **Regularized Discriminant Analysis (RDA)** applies shrinkage to covariance estimates:

Definition 7.1 (Friedman’s Shrinkage Covariance Estimator).

$$\hat{\Sigma}_{\text{reg}}(\gamma) = (1 - \gamma)\hat{\Sigma} + \gamma\sigma^2\mathbf{I}_d \quad (46)$$

where $\gamma \in [0, 1]$ is shrinkage parameter, and $\sigma^2 = \frac{1}{d}\text{Tr}(\hat{\Sigma})$ is average variance.

Shrinkage adds a positive diagonal bias, shifting zero eigenvalues to positive values $\gamma\sigma^2 > 0$. This ensures $\hat{\Sigma}_{\text{reg}}$ is strictly positive definite and invertible.

8 Production-Grade Vectorized NumPy Implementation

The following Python class implements Linear Discriminant Analysis for classification and dimensionality reduction, including shrinkage regularization.

```
1 import numpy as np
2
3 class LinearDiscriminantAnalysisEngine:
4     """
5     Production-grade Linear Discriminant Analysis (LDA) engine supporting
6     multiclass classification, dimensionality reduction, and shrinkage
7     regularization.
8     """
9     def __init__(self, n_components: int = None, shrinkage: float = 0.0):
10         self.n_components = n_components
11         self.shrinkage = shrinkage
12
13         self.priors_ = None          # Class priors pi_k (K,)
14         self.means_ = None           # Class means mu_k (K, d)
15         self.covariance_ = None      # Pooled covariance Sigma (d, d)
16         self.scalings_ = None        # Projection eigenvectors W (d, p)
17         self.classes_ = None         # Unique class labels
18         self.coef_ = None            # Linear discriminant weights (K, d)
19         self.intercept_ = None       # Linear discriminant biases (K,)
20
21     def fit(self, X: np.ndarray, y: np.ndarray):
22         """Fits LDA model parameters via Maximum Likelihood Estimation."""
23         N, d = X.shape
24         self.classes_ = np.unique(y)
25         K = len(self.classes_)
26
27         # 1. Parameter Initialization
28         self.priors_ = np.zeros(K)
29         self.means_ = np.zeros((K, d))
30         S_W = np.zeros((d, d))
31
32         global_mean = np.mean(X, axis=0)
33
34         # 2. Compute Class Statistics and Within-Class Scatter S_W
35         for k_idx, k in enumerate(self.classes_):
36             X_k = X[y == k]
37             N_k = X_k.shape[0]
38
39             self.priors_[k_idx] = N_k / N
```

```

39         self.means_[k_idx] = np.mean(X_k, axis=0)
40
41         # Center data for class k
42         X_k_centered = X_k - self.means_[k_idx]
43         S_W += X_k_centered.T @ X_k_centered
44
45         # 3. Pooled Covariance Matrix with Shrinkage Regularization
46         unbiased_scale = 1.0 / (N - K) if N > K else 1.0 / N
47         Sigma = unbiased_scale * S_W
48
49         if self.shrinkage > 0.0:
50             gamma = np.clip(self.shrinkage, 0.0, 1.0)
51             avg_var = np.trace(Sigma) / d
52             Sigma = (1.0 - gamma) * Sigma + gamma * avg_var * np.eye(d)
53
54         self.covariance_ = Sigma
55         Sigma_inv = np.linalg.pinv(Sigma)
56
57         # 4. Linear Discriminant Parameters:  $w_k = \text{Sigma}^{-1} * \mu_k$ 
58         self.coef_ = self.means_ @ Sigma_inv # (K, d)
59         self.intercept_ = -0.5 * np.sum(self.means_ * self.coef_, axis=1) +
        np.log(self.priors_) # (K,)
60
61         # 5. Multiclass Dimensionality Reduction Subspace (Eigen-
        decomposition of  $S_W^{-1} * S_B$ )
62         S_B = np.zeros((d, d))
63         for k_idx in range(K):
64             mean_diff = (self.means_[k_idx] - global_mean).reshape(-1, 1)
65             S_B += (self.priors_[k_idx] * N) * (mean_diff @ mean_diff.T)
66
67         # Solve Generalized Eigenvalue Problem:  $S_W^{-1} * S_B * w = \lambda * w$ 
68         eigvals, eigvecs = np.linalg.eig(Sigma_inv @ (S_B / N))
69
70         # Sort eigenvectors by descending real eigenvalues
71         sorted_indices = np.argsort(np.real(eigvals))[:, :-1]
72         eigvecs_sorted = np.real(eigvecs[:, sorted_indices])
73
74         max_components = min(d, K - 1)
75         p = self.n_components if self.n_components else max_components
76         p = min(p, max_components)
77
78         self.scalings_ = eigvecs_sorted[:, :p]
79         return self
80
81     def decision_function(self, X: np.ndarray) -> np.ndarray:
82         """Evaluates linear discriminant functions  $\delta_k(X) = X * w_k + b_k$ ."""
83         return X @ self.coef_.T + self.intercept_
84
85     def predict_proba(self, X: np.ndarray) -> np.ndarray:
86         """Computes posterior class probabilities  $P(Y=k|X)$  using Softmax."""
87
88         deltas = self.decision_function(X)
89         # Numerical stability shift
90         shifted_deltas = deltas - np.max(deltas, axis=1, keepdims=True)
91         exp_deltas = np.exp(shifted_deltas)
92         return exp_deltas / np.sum(exp_deltas, axis=1, keepdims=True)
93
94     def predict(self, X: np.ndarray) -> np.ndarray:

```

```

94         """Predicts class labels corresponding to maximum discriminant
function."""
95         probabilities = self.predict_proba(X)
96         max_indices = np.argmax(probabilities, axis=1)
97         return self.classes_[max_indices]
98
99     def transform(self, X: np.ndarray) -> np.ndarray:
100         """Projects data onto leading K-1 discriminative feature axes."""
101         if self.scalings_ is None:
102             raise ValueError("Model must be fitted before calling transform
.")
103         return X @ self.scalings_
104
105
106 if __name__ == "__main__":
107     np.random.seed(42)
108
109     # Generate Synthetic 3-Class Gaussian Dataset (4 features)
110     N_per_class = 100
111     d_features = 4
112
113     mu0 = np.array([-2.0, -2.0, 1.0, 0.0])
114     mu1 = np.array([2.0, 2.0, -1.0, 1.0])
115     mu2 = np.array([0.0, 0.0, 3.0, -2.0])
116
117     cov_shared = np.array([[1.0, 0.3, 0.2, 0.0],
118                             [0.3, 1.2, 0.1, 0.1],
119                             [0.2, 0.1, 0.9, 0.2],
120                             [0.0, 0.1, 0.2, 1.1]])
121
122     X0 = np.random.multivariate_normal(mu0, cov_shared, N_per_class)
123     X1 = np.random.multivariate_normal(mu1, cov_shared, N_per_class)
124     X2 = np.random.multivariate_normal(mu2, cov_shared, N_per_class)
125
126     X_data = np.vstack([X0, X1, X2])
127     y_data = np.hstack([np.zeros(N_per_class), np.ones(N_per_class), 2 * np
.ones(N_per_class)]).astype(int)
128
129     # Train LDA Engine
130     lda = LinearDiscriminantAnalysisEngine(n_components=2, shrinkage=0.05)
131     lda.fit(X_data, y_data)
132
133     # Predict and Project
134     y_preds = lda.predict(X_data)
135     X_projected = lda.transform(X_data)
136
137     accuracy = np.mean(y_preds == y_data)
138     print(f"== LDA CLASSIFICATION ACCURACY: {accuracy * 100:.2f}% ==")
139     print(f"Original Feature Shape : {X_data.shape}")
140     print(f"Projected Subspace Shape: {X_projected.shape}")

```

Listing 1: Vectorized NumPy implementation of Linear Discriminant Analysis.